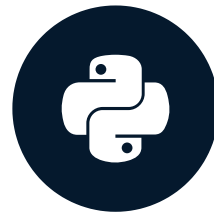


# Khám phá mô hình thưởng

REINFORCEMENT LEARNING FROM HUMAN FEEDBACK (RLHF)



**Mina Parham**  
AI Engineer

# Quy trình đến nay

Prompt dataset



Prompt



Initial LLM



Initial response

Preference dataset



# Quy trình đến nay

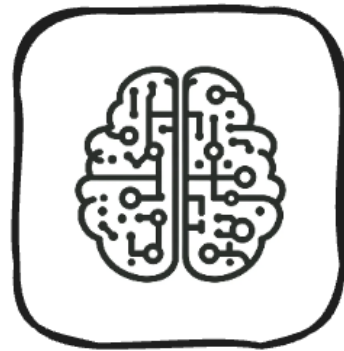
Prompt dataset



Prompt

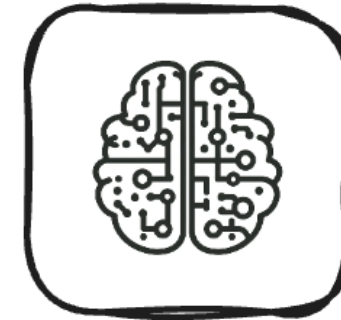


Initial LLM



Initial response

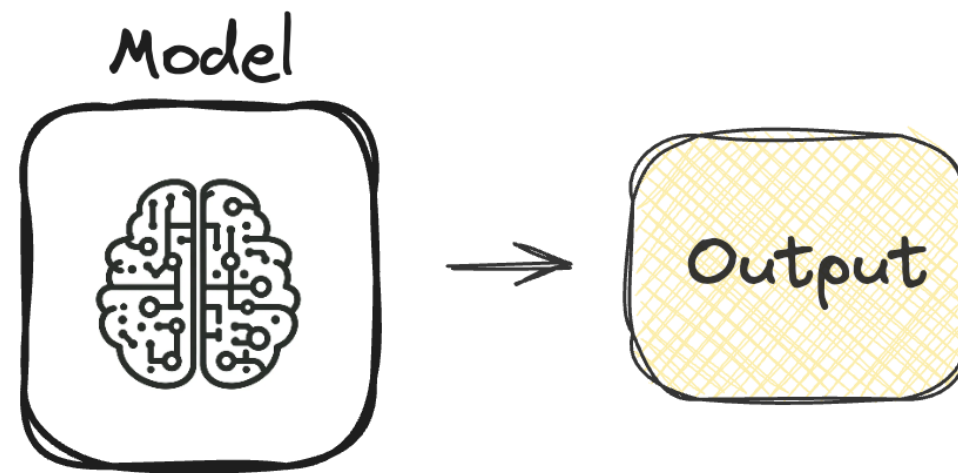
Reward model



Preference dataset

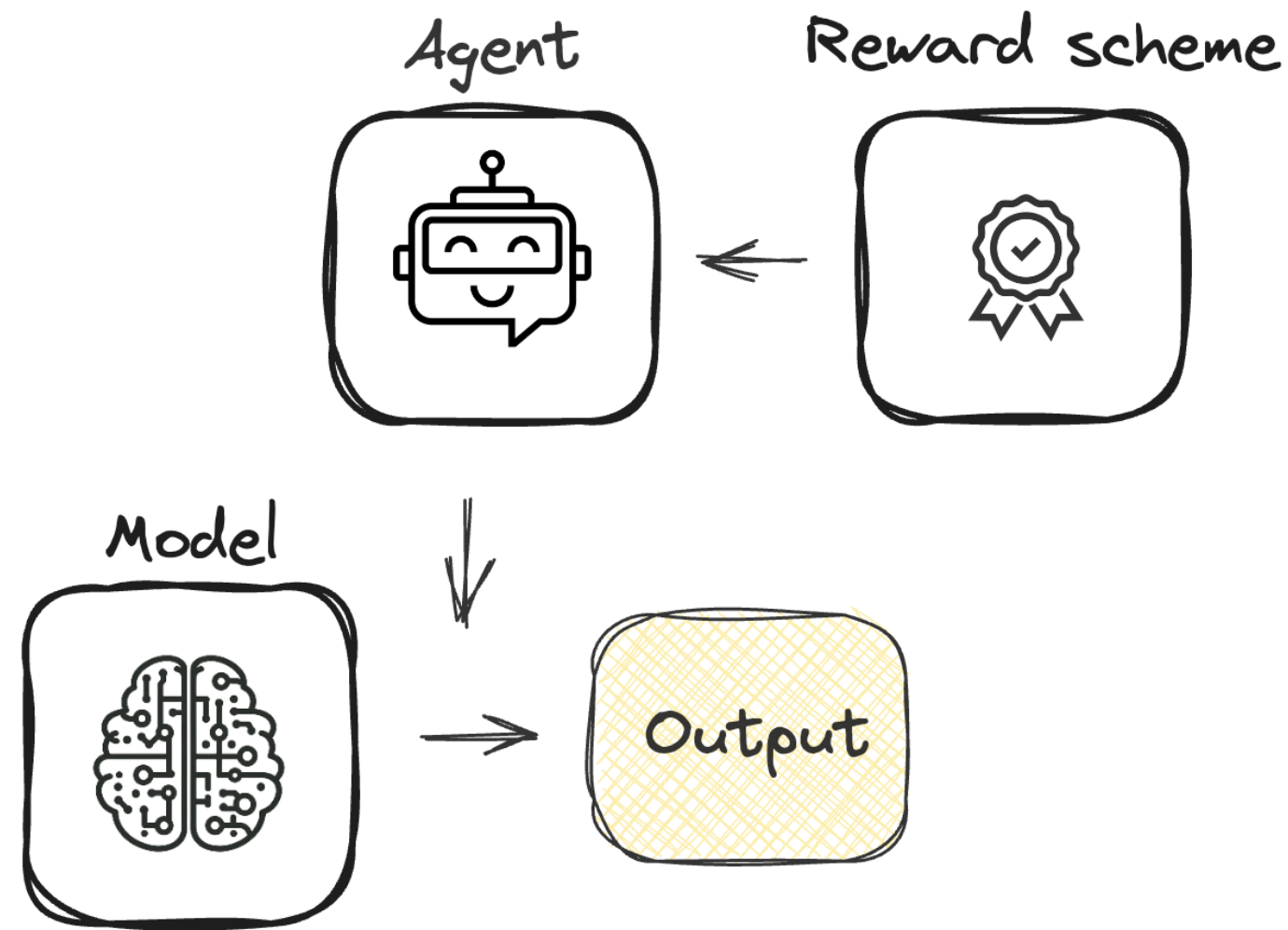


# Mô hình thưởng là gì?



# Mô hình thưởng là gì?

- Mô hình cung cấp thông tin cho tác tử
- Tác tử đánh giá mô hình để tối đa hóa phần thưởng



# Dùng RewardTrainer

```
from trl import RewardTrainer, RewardConfig
from transformers import AutoModelForSequenceClassification, AutoTokenizer
from datasets import load_dataset
```

```
# Load pre-trained model and tokenizer
model = AutoModelForSequenceClassification.from_pretrained("gpt2", num_labels=1)
tokenizer = AutoTokenizer.from_pretrained("gpt2")
# Load dataset in the required format
dataset = load_dataset("path/to/dataset")
```

# Huấn luyện mô hình thưởng

```
# Define training arguments
training_args = RewardConfig(
    output_dir="path/to/output/dir",
    per_device_train_batch_size=8,
    per_device_eval_batch_size=8,
    num_train_epochs=3,
    learning_rate=1e-3
)
```

# Huấn luyện mô hình thưởng

```
# Initialize the RewardTrainer
trainer = RewardTrainer(
    model=model,
    args=training_args,
    train_dataset=dataset["train"],
    eval_dataset=dataset["validation"],
    tokenizer=tokenizer,
)
```

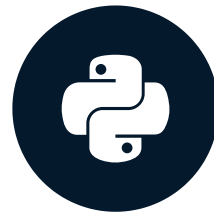
```
# Train the reward model
trainer.train()
```

# Ayo berlatih!

REINFORCEMENT LEARNING FROM HUMAN FEEDBACK (RLHF)

# Tinh chỉnh hiệu quả trong RLHF

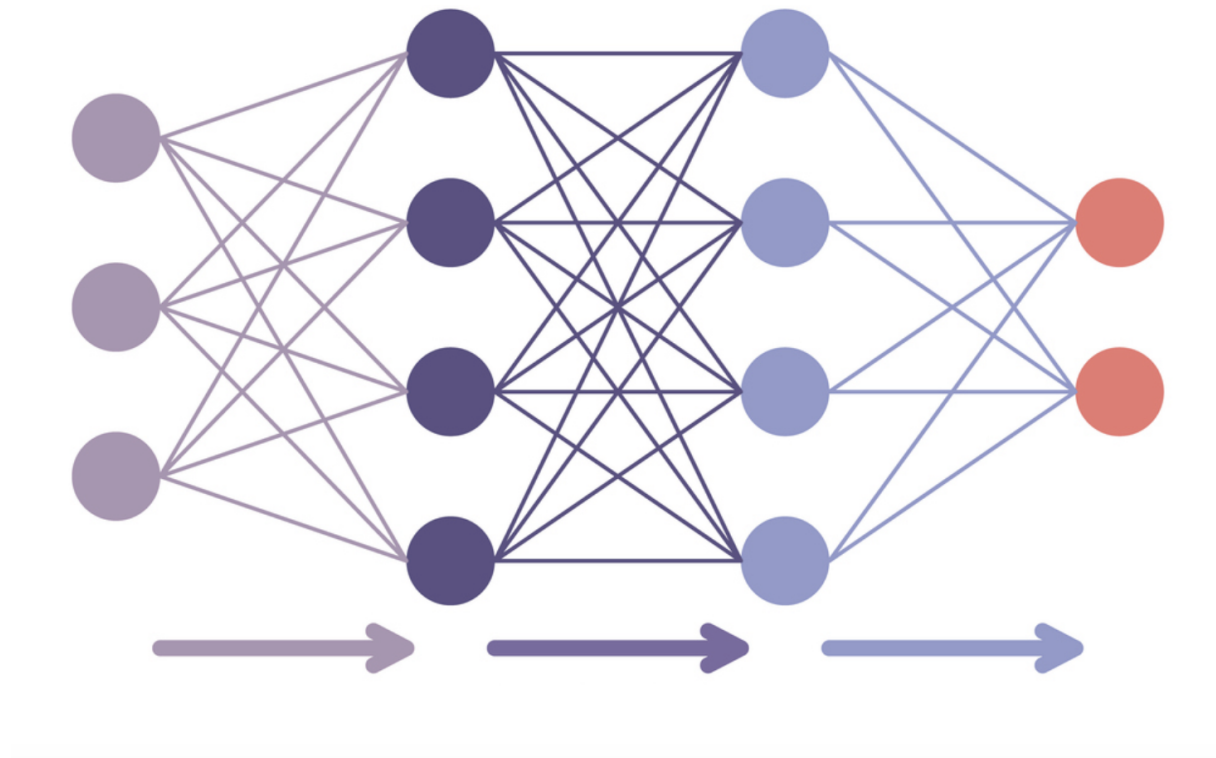
REINFORCEMENT LEARNING FROM HUMAN FEEDBACK (RLHF)



**Mina Parham**  
AI Engineer

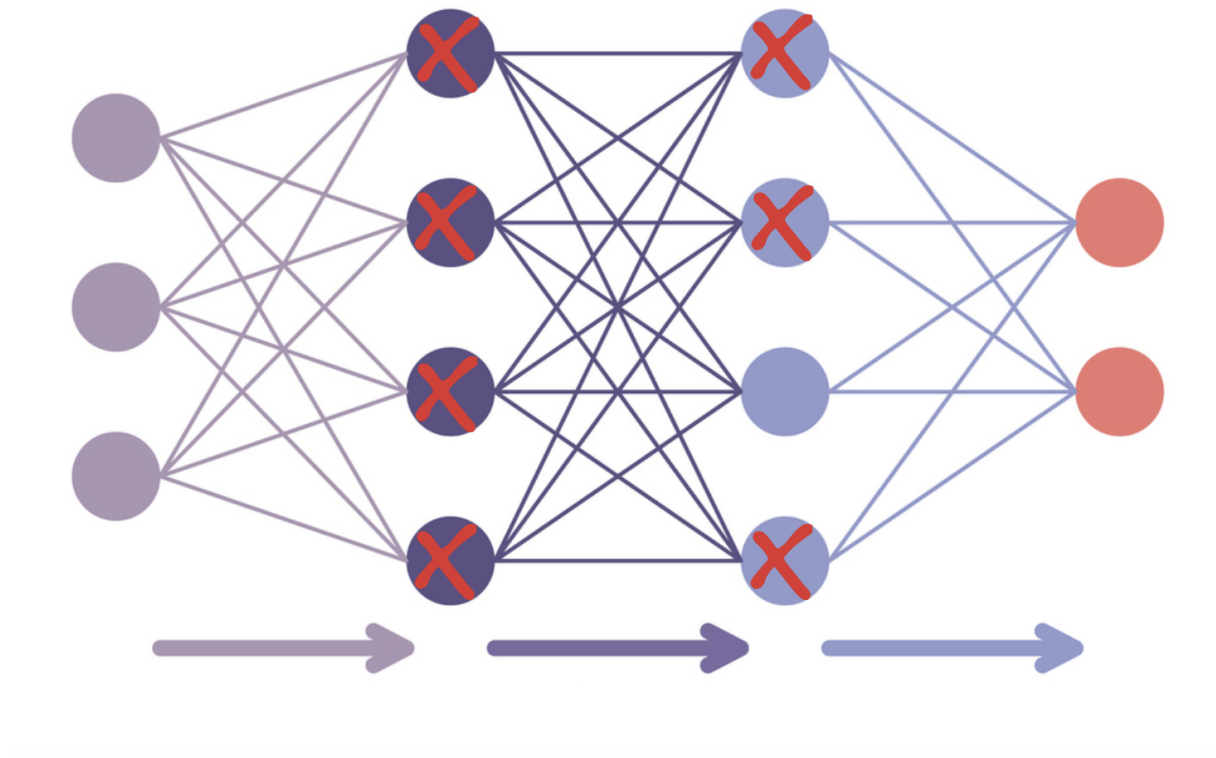
# Tinh chỉnh tiết kiệm tham số

- Tinh chỉnh toàn bộ mô hình



# Tinh chỉnh tiết kiệm tham số

- Tinh chỉnh với PEFT



- LoRA: chỉ điều chỉnh một vài lớp
- Lượng tử hóa: giảm độ chính xác kiểu dữ liệu

# Bước 1: tải mô hình hoạt động ở độ chính xác 8-bit

```
from peft import prepare_model_for_int8_training

pretrained_model = AutoModelForCausalLM.from_pretrained(
    model_name,
    load_in_8bit=True
)

pretrained_model_8bit = prepare_model_for_int8_training(pretrained_model)
```

## Bước 2: thêm adapter có thể huấn luyện bằng peft

```
from peft import LoraConfig, get_peft_model

config = LoraConfig(
    r=32, # Hạng của ma trận hạng thấp
    lora_alpha=32, # Hệ số nhân cho cập nhật LoRA
    lora_dropout=0.1, # Tỷ lệ dropout cho các lớp LoRA
    bias="lora_only" # Chỉ cập nhật bias cho lớp LoRA, phần khác giữ nguyên
)

lora_model = get_peft_model(pretrained_model_8bit, config)
model = AutoModelForCausalLMWithValueHead.from_pretrained(lora_model)
```

# Bước 3: dùng một mô hình cho logits tham chiếu và hoạt động

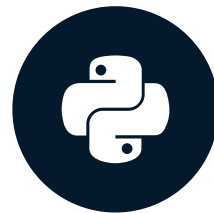
```
ppo_trainer = PPOTrainer(  
    config, # Cấu hình vừa định nghĩa  
    model, # Mô hình PPO của chúng ta  
    ref_model=None,  
    tokenizer=tokenizer,  
    dataset=dataset,  
    data_collator=collator,  
    optimizer=optimizer  
)
```

# Ayo berlatih!

REINFORCEMENT LEARNING FROM HUMAN FEEDBACK (RLHF)

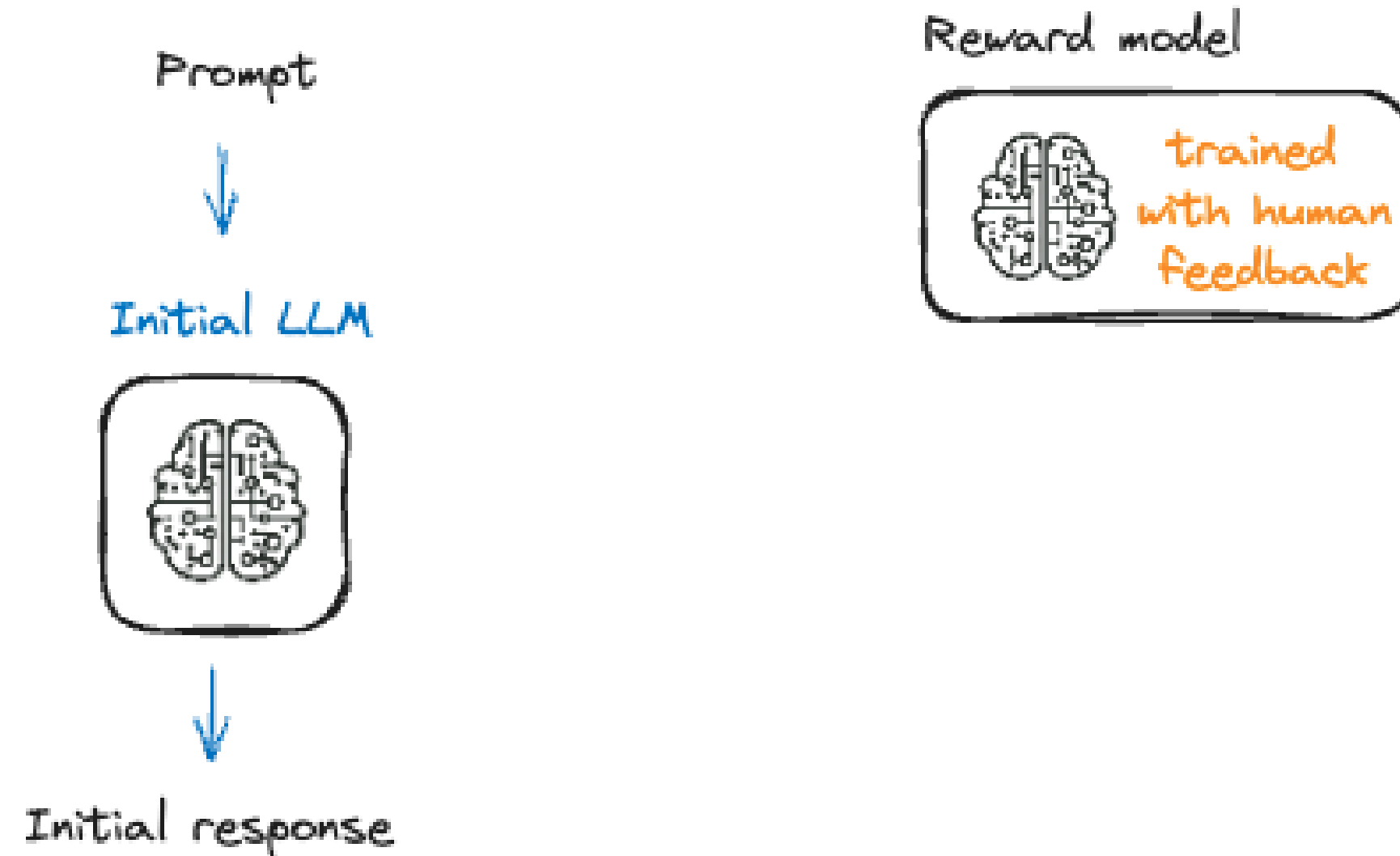
# Huấn luyện với PPO

REINFORCEMENT LEARNING FROM HUMAN FEEDBACK (RLHF)

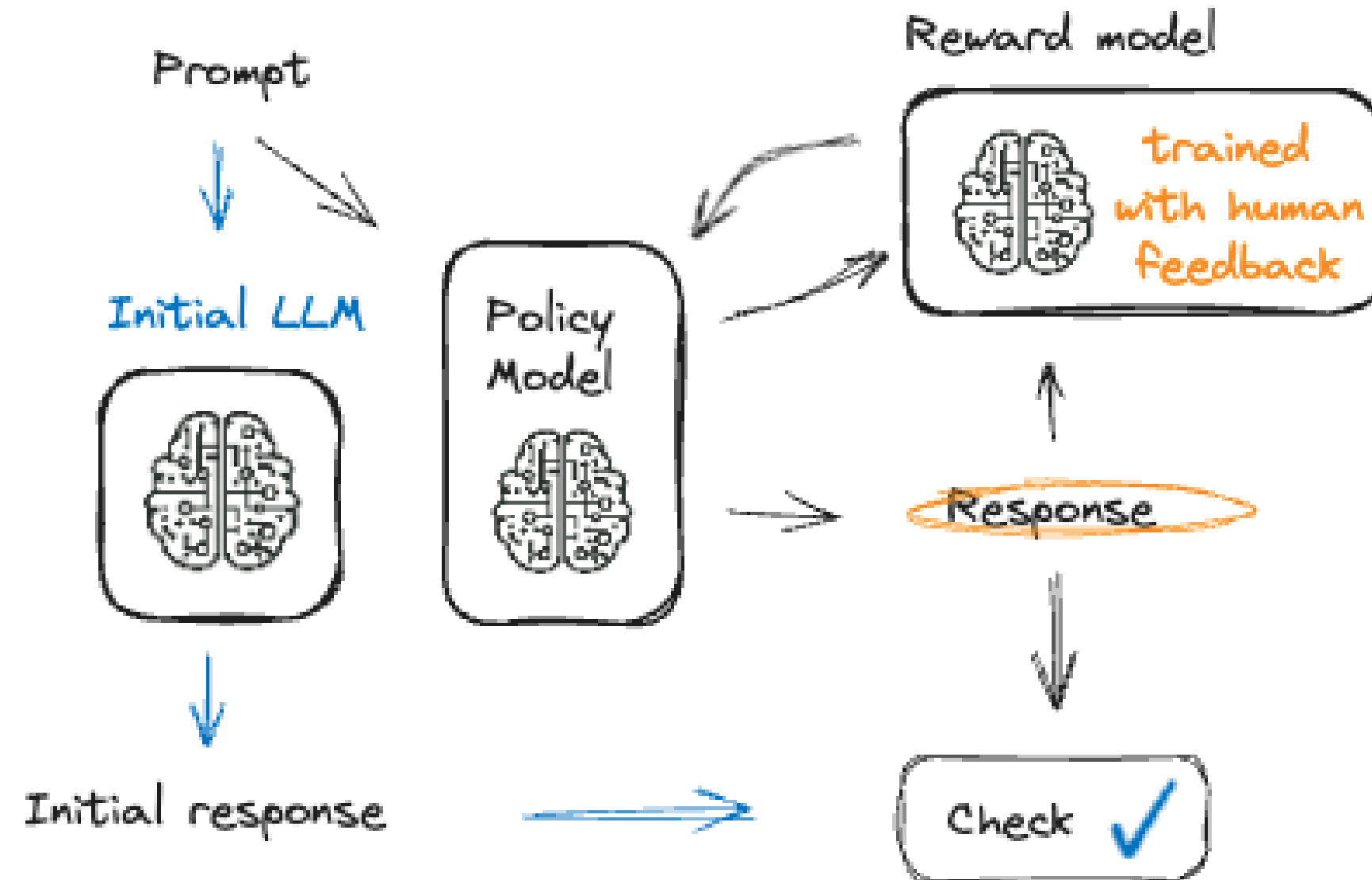


**Mina Parham**  
AI Engineer

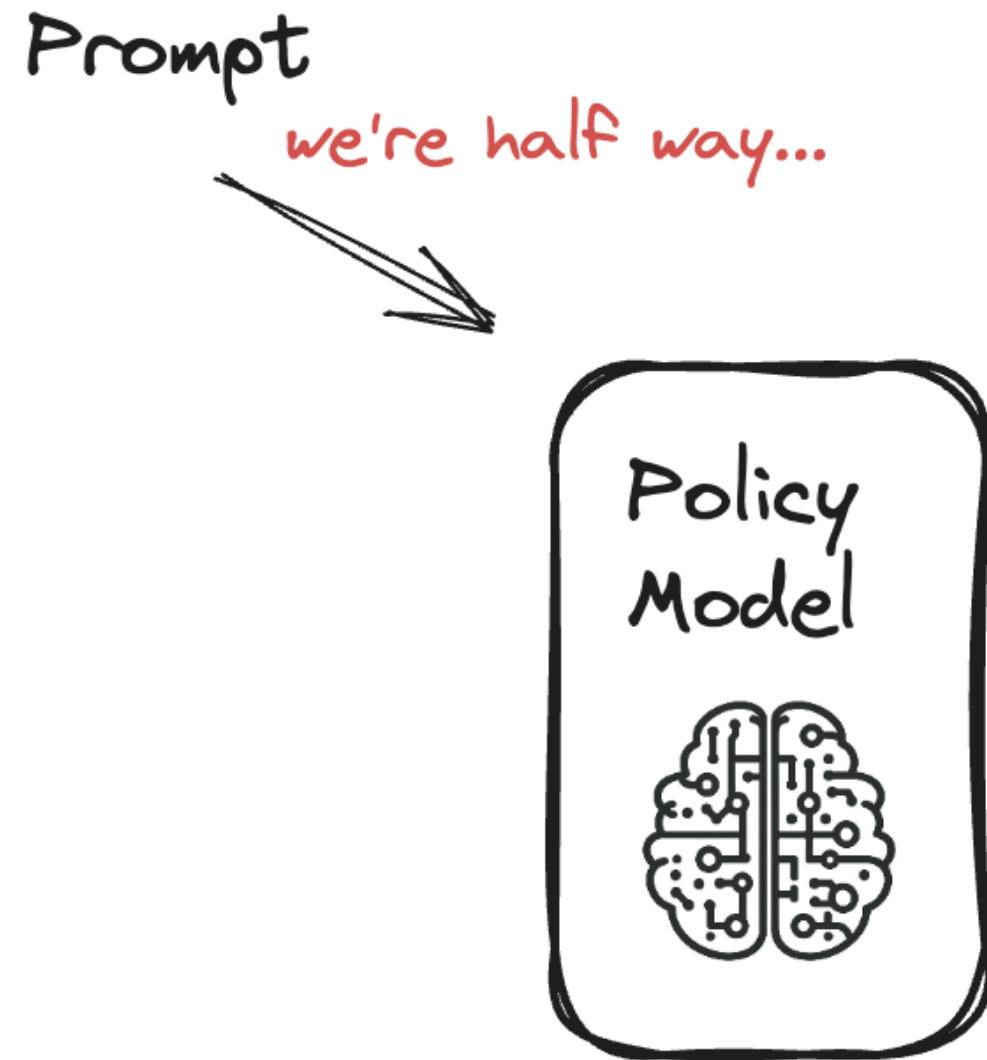
# Fine-tune với học tăng cường



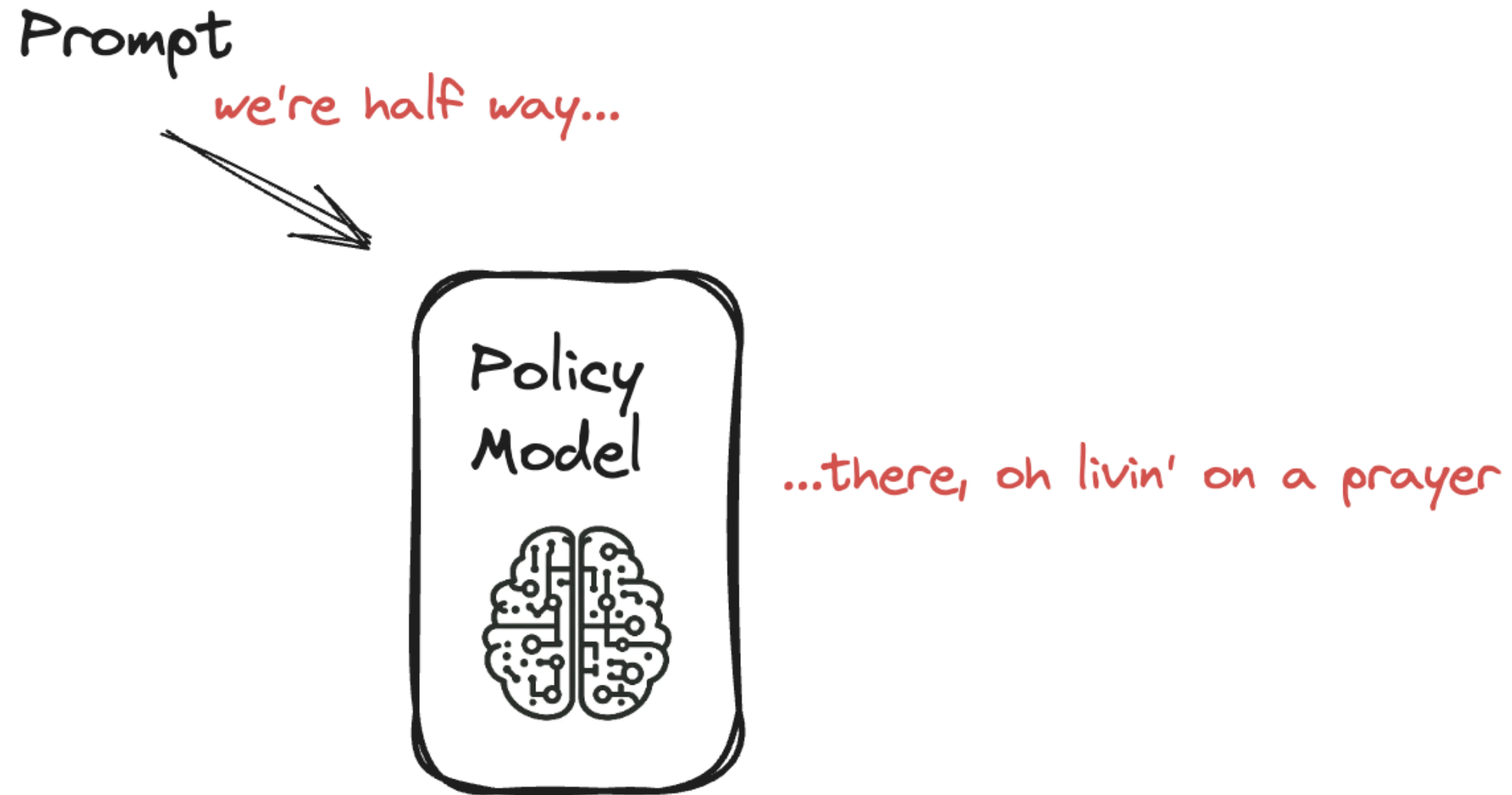
# Fine-tune với học tăng cường



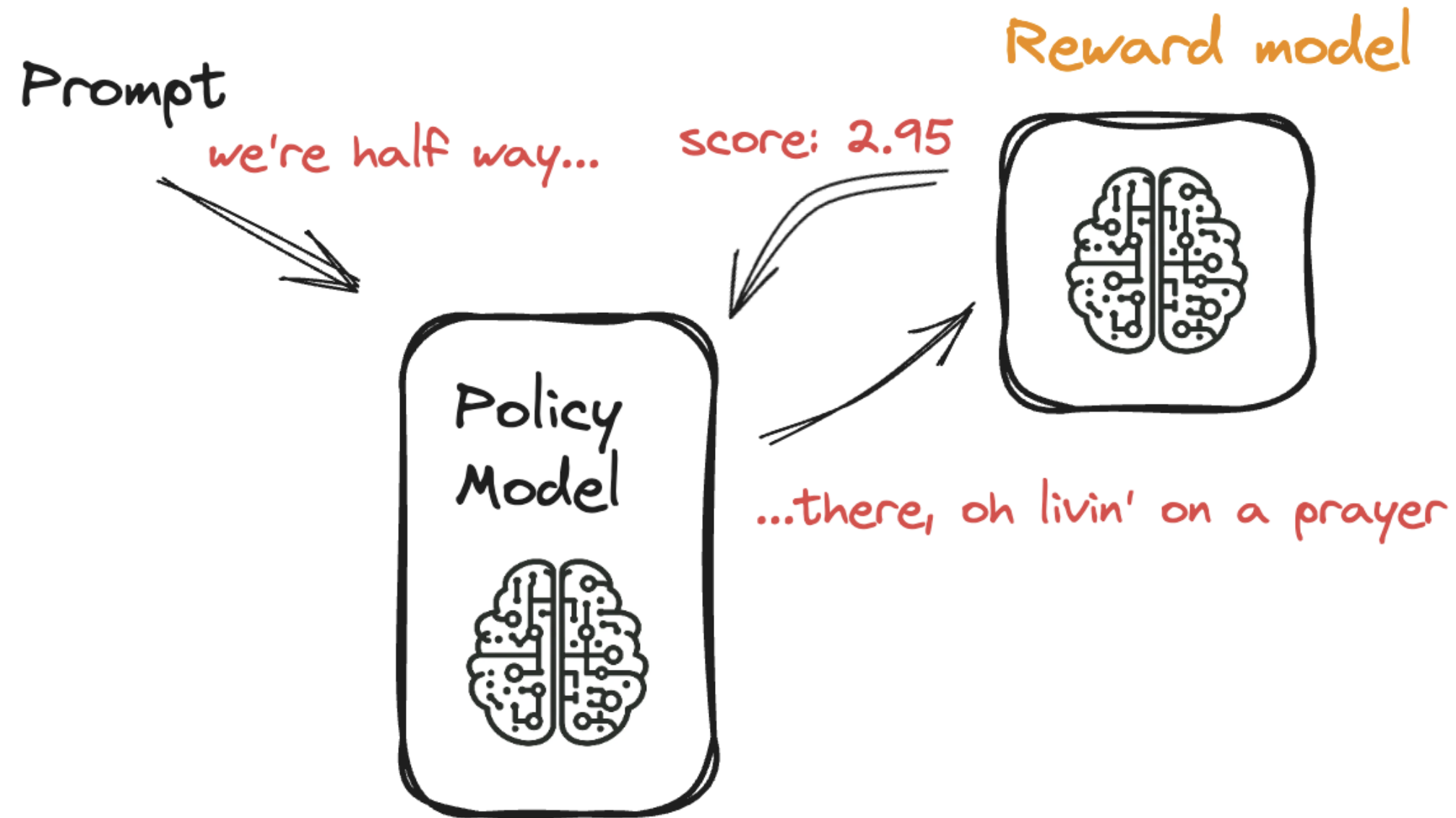
# Fine-tune mô hình ngôn ngữ với PPO



# Fine-tune mô hình ngôn ngữ với PPO



# Fine-tune mô hình ngôn ngữ với PPO



# Fine-tune mô hình ngôn ngữ với PPO

- PPO: điều chỉnh mô hình dần dần
- Tránh khớp quá mức theo phản hồi



# Triển khai PPOTrainer với TRL

```
from trl import PPOConfig  
config = PPOConfig(model_name="gpt2", learning_rate=1.4e-5)
```

```
from trl import AutoModelForCausalLMWithValueHead  
model = AutoModelForCausalLMWithValueHead.from_pretrained(config.model_name)  
tokenizer = AutoTokenizer.from_pretrained(config.model_name)
```

```
from trl import PPOTrainer  
ppo_trainer = PPOTrainer(model=model, config=config, dataset=dataset,  
                        tokenizer=tokenizer)
```

# Bắt đầu vòng lặp huấn luyện

```
for epoch in tqdm(range(10), "epoch: "):  
  
    for batch in tqdm(ppo_trainer.data_loader):  
        # Get responses  
        response_tensors = ppo_trainer.generate(batch["input_ids"])  
        batch["response"] = [tokenizer.decode(r.squeeze()) for r in response_tensors]  
        # Compute reward score  
        texts = [q + r for q, r in zip(batch["query"], batch["response"])]  
        rewards = reward_model(texts)  
        stats = ppo_trainer.step(query_tensors, response_tensors, rewards)  
        ppo_trainer.log_stats(stats, batch, rewards)
```

# Ayo berlatih!

REINFORCEMENT LEARNING FROM HUMAN FEEDBACK (RLHF)